

Samostalna vježba: Mood Tracker aplikacija

Naziv projekta

08_React/04_Practice_MoodTracker/

Cilj vježbe

U ovoj vježbi učenici će izraditi malu React aplikaciju koja prati raspoloženje korisnika tijekom dana.

Aplikacija će imati nekoliko gumba pomoću kojih korisnik može povećavati ili smanjivati svoj "mood score", resetirati vrijednost i vidjeti poruku koja se mijenja ovisno o trenutnom rezultatu.

Ova vježba koristi koncepte koje su učenici već prošli:

- React komponentu
- JSX
- `className`
- event handler preko `onClick`
- `useState`
- funkcijski updater oblik: `prev => prev + 1`
- conditional rendering pomoću ternarnog operatora
- conditional rendering pomoću `&&`
- osnovni CSS

U ovoj vježbi **ne koristimo props, array `.map()`, forme, `useEffect`, local storage ni više komponenti u zasebnim datotekama**, osim ako su ti koncepti već posebno obrađeni.

Scenarij

Zamislite da izrađujete jednostavnu aplikaciju koja korisniku pomaže pratiti svoje raspoloženje.

Korisnik počinje s vrijednošću `0`.

Može kliknuti gumbe:

Better mood +1
Great moment +5
Worse mood -1
Reset

Na temelju trenutnog broja aplikacija treba prikazati drugačiju poruku.

Ako je mood score pozitivan, aplikacija prikazuje ohrabrujuću poruku.

Ako je mood score jednak nuli, aplikacija prikazuje neutralnu poruku.

Ako je mood score negativan, aplikacija prikazuje poruku podrške.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest mood-tracker -- --template react
```

Zatim uđite u projekt:

```
cd mood-tracker
```

Instalirajte potrebne pakete:

```
npm install
```

Pokrenite development server:

```
npm run dev
```

Otvorite aplikaciju u pregledniku.

2. Očistite početni projekt

U datoteci `src/App.jsx` obrišite početni sadržaj i pripremite osnovnu strukturu aplikacije.

Početna komponenta može izgledati ovako:

```
import './App.css';

function App() {
  return (
    <main className="app">
      <h1>Mood Tracker</h1>
    </main>
  );
}

export default App;
```

U datoteci `src/App.css` obrišite postojeći CSS kako biste mogli sami urediti aplikaciju.

3. Dodajte state za mood score

Na vrhu datoteke `App.jsx` importajte `useState` iz Reacta:

```
import { useState } from "react";
import './App.css';
```

Unutar `App` komponente dodajte state varijablu:

```
const [moodScore, setMoodScore] = useState(0);
```

Ova vrijednost predstavlja trenutno raspoloženje korisnika.

Početna vrijednost je `0`, što znači neutralno raspoloženje.

4. Prikažite trenutni mood score

U JSX-u prikažite trenutnu vrijednost statea.

Stranica treba prikazivati nešto poput:

Current mood score: 0

Primjer:

```
<p className="score">Current mood score: {moodScore}</p>
```

Kada se state kasnije promijeni, ova vrijednost na ekranu treba se automatski ažurirati.

5. Dodajte gumbе za promjenu statea

Dodajte četiri gumba:

Better mood +1
Great moment +5
Worse mood -1
Reset

Svaki gumb treba imati `onClick`.

Za gumbе koji mijenjaju vrijednost na temelju prethodne vrijednosti koristite funkcijski updater.

Primjer:

```
<button onClick={() => setMoodScore(prev => prev + 1)}>  
  Better mood +1  
</button>
```

Za resetiranje možete izravno postaviti vrijednost na `0`:

```
<button onClick={() => setMoodScore(0)}>  
  Reset  
</button>
```

Važan cilj ove vježbe je da učenici shvate razliku između ova dva pristupa.

Kada nova vrijednost ovisi o staroj, koristimo:

```
setMoodScore(prev => prev + 1)
```

Kada nova vrijednost ne ovisi o staroj, možemo koristiti:

```
setMoodScore(0)
```

6. Dodajte poruku pomoću ternarnog operatora

Ispod broja dodajte poruku koja se mijenja ovisno o vrijednosti `moodScore`.

Pravila su sljedeća:

Ako je `moodScore` veći od 0, prikazite:

You are having a positive day!

Ako je `moodScore` jednak 0, prikazite:

Your mood is neutral right now.

Ako je `moodScore` manji od 0, prikazite:

Take a deep breath. Tomorrow can be better.

Budući da imamo tri mogućnosti, možete koristiti ugniježđeni ternarni operator:

```
<p className="message">
  {moodScore > 0
    ? "You are having a positive day!"
    : moodScore === 0
      ? "Your mood is neutral right now."
      : "Take a deep breath. Tomorrow can be better."}
</p>
```

Ovdje je važno učenicima objasniti da ternarni operator vraća vrijednost. Zato ga možemo koristiti unutar JSX-a.

7. Dodajte posebnu poruku pomoću `&&` operatora

Dodajte posebnu poruku koja se prikazuje samo ako je `moodScore` veći ili jednak 10.

Poruka:

Amazing! This looks like an excellent day.

Koristite `&&` operator:

```
{moodScore >= 10 && (  
  <p className="bonus-message">  
    Amazing! This looks like an excellent day.  
  </p>  
)}
```

Ova poruka se ne treba prikazivati kada je `moodScore` manji od `10`.

8. Dodajte ograničenje za negativne vrijednosti

Dodajte još jednu posebnu poruku koja se prikazuje samo ako je `moodScore` manji ili jednak `-5`.

Poruka:

It may be a good moment to take a short break.

Primjer:

```
{moodScore <= -5 && (  
  <p className="support-message">  
    It may be a good moment to take a short break.  
  </p>  
)}
```

Ovdje učenici dodatno vježbaju uvjetni prikaz pomoću `&&`.

9. Stilizirajte aplikaciju

U datoteci `src/App.css` uredite aplikaciju tako da izgleda kao mala kartica.

Dodajte:

- centriran sadržaj
- maksimalnu širinu
- pozadinu kartice
- gumbe u redu
- razmake između elemenata
- različit stil za glavnu poruku, bonus poruku i poruku podrške

Primjer CSS-a:

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  background-color: #eef2f7;  
}
```

```
.app {  
  max-width: 520px;  
  margin: 60px auto;  
  padding: 32px;  
  text-align: center;  
  background-color: white;  
  border-radius: 16px;  
  border: 1px solid #d8dee9;  
}
```

```
h1 {  
  margin-top: 0;  
}
```

```
.score {  
  font-size: 28px;  
  font-weight: bold;  
  margin: 24px 0;  
}
```

```
.message {  
  font-size: 18px;  
  margin-bottom: 24px;  
}
```

```
.button-group {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 12px;  
  justify-content: center;  
}
```

```
button {  
  padding: 10px 16px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  background-color: #f7f9fc;  
  cursor: pointer;  
  font-size: 15px;  
}
```

```
button:hover {
```

```
background-color: #e9eef6;
}
```

```
.bonus-message {
  margin-top: 20px;
  color: #176b3a;
  font-weight: bold;
}
```

```
.support-message {
  margin-top: 20px;
  color: #9a5b00;
  font-weight: bold;
}
```

Očekivano ponašanje aplikacije

Kada se aplikacija prvi put otvori, prikazuje:

Mood Tracker
Current mood score: 0
Your mood is neutral right now.

Kada korisnik klikne **Better mood +1**, vrijednost se poveća za 1.

Kada korisnik klikne **Great moment +5**, vrijednost se poveća za 5.

Kada korisnik klikne **Worse mood -1**, vrijednost se smanji za 1.

Kada korisnik klikne **Reset**, vrijednost se vrati na 0.

Ako vrijednost dođe do 10 ili više, prikazuje se dodatna pozitivna poruka.

Ako vrijednost padne na -5 ili manje, prikazuje se dodatna poruka podrške.

Puno rješenje

src/App.jsx

```
import { useState } from "react";
```



```

import './App.css';

function App() {
  const [moodScore, setMoodScore] = useState(0);

  return (
    <main className="app">
      <h1>Mood Tracker</h1>

      <p className="score">Current mood score: {moodScore}</p>

      <p className="message">
        {moodScore > 0
          ? "You are having a positive day!"
          : moodScore === 0
            ? "Your mood is neutral right now."
            : "Take a deep breath. Tomorrow can be better."}
      </p>

      <div className="button-group">
        <button onClick={() => setMoodScore(prev => prev + 1)}>
          Better mood +1
        </button>

        <button onClick={() => setMoodScore(prev => prev + 5)}>
          Great moment +5
        </button>

        <button onClick={() => setMoodScore(prev => prev - 1)}>
          Worse mood -1
        </button>

        <button onClick={() => setMoodScore(0)}>
          Reset
        </button>
      </div>

      {moodScore >= 10 && (
        <p className="bonus-message">
          Amazing! This looks like an excellent day.
        </p>
      )}

      {moodScore <= -5 && (
        <p className="support-message">
          It may be a good moment to take a short break.
        </p>
      )}
    </main>
  );
}

```

```
    </main>
  );
}

export default App;
```

src/App.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 520px;
  margin: 60px auto;
  padding: 32px;
  text-align: center;
  background-color: white;
  border-radius: 16px;
  border: 1px solid #d8dee9;
}

h1 {
  margin-top: 0;
}

.score {
  font-size: 28px;
  font-weight: bold;
  margin: 24px 0;
}

.message {
  font-size: 18px;
  margin-bottom: 24px;
}

.button-group {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  justify-content: center;
}
```

```
button {
  padding: 10px 16px;
  border: 1px solid #ccd3df;
  border-radius: 8px;
  background-color: #f7f9fc;
  cursor: pointer;
  font-size: 15px;
}

button:hover {
  background-color: #e9eef6;
}

.bonus-message {
  margin-top: 20px;
  color: #176b3a;
  font-weight: bold;
}

.support-message {
  margin-top: 20px;
  color: #9a5b00;
  font-weight: bold;
}
```

Dodatni izazovi

Izazov 1: Dodajte gumb **Bad day -5**

Dodajte novi gumb koji smanjuje `moodScore` za 5.

Gumb treba koristiti funkcijski updater:

```
<button onClick={() => setMoodScore(prev => prev - 5)}>
  Bad day -5
</button>
```

Izazov 2: Prikažite tekstualni status

Dodajte novi paragraf koji prikazuje status:

Status: Positive
Status: Neutral
Status: Negative

Status treba ovisiti o vrijednosti `moodScore`.

Primjer:

```
<p>  
  Status:{" "  
  {moodScore > 0  
    ? "Positive"  
    : moodScore === 0  
      ? "Neutral"  
      : "Negative"}  
</p>
```

Izazov 3: Dodajte gumb koji testira funkcijski updater

Dodajte gumb:

Triple boost +3

Taj gumb treba pozvati setter tri puta zaredom:

```
<button  
  onClick={() => {  
    setMoodScore(prev => prev + 1);  
    setMoodScore(prev => prev + 1);  
    setMoodScore(prev => prev + 1);  
  }}  
>  
  Triple boost +3  
</button>
```